THEORY · PYTHON · NUM-1.1 · WEEK 2

Floating-point numbers and machine epsilon

The foundation under every NUM topic. Tao Pang's chapter 1 gives the background but is brief on number storage, so this note stands on its own.

How a float is stored

A 64-bit float ("double", Python's `float`) is scientific notation in base 2:

$$x = \pm 1.b_1 b_2 \ldots b_{52} \times 2^{e}$$

There are **52 bits** after the binary point (the mantissa) and an exponent e from about -1022 to 1023. So every float has the same *relative* precision, about 16 decimal digits, whether it is 10^{-10} or 10^{20}.

Machine epsilon

The **machine epsilon** ε is the gap between 1.0 and the next float above it. Anything much smaller than ε added to 1.0 is rounded away. With m mantissa bits, $\varepsilon = 2^{-m}$. Near any other number x the gap scales with x:

$$\text{gap}(x) \approx \varepsilon\,|x| \qquad (\text{between } \tfrac{1}{2}\varepsilon|x| \text{ and } \varepsilon|x|)$$

A sum $x + a$ is rounded to the nearest float, so if $|a|$ is below half the gap, $x + a$ comes out equal to x.

Worked example (different numbers, same idea)

Picture a calculator that keeps **4 significant digits**. Then $1 + 0.001 = 1.001$ is stored fine, but $1 + 0.0001 = 1.0001$ is rounded to 1.000: its epsilon is about 10^{-3}. Near 50 000 (stored as 5.000×10^4) the last digit is worth 10, so $50\,000 + 1$ gives 50 000 again. The relative gap stays about ε; the absolute gap grows with the number. (32-bit floats, common on GPUs, have $m = 23$: $\varepsilon \approx 1.19 \times 10^{-7}$.)

Overflow and underflow

- The largest double is about 1.80×10^{308}. Beyond it you get **overflow**: `1e308 * 10` gives `inf`.
- The smallest normal double is about 2.23×10^{-308}. Far below it you get **underflow** to zero: `np.exp(-800.0)` gives `0.0`.
- Large intermediate powers can overflow even when the answer fits: rescale (densities in units of 10^{20} m^{-3}) or use logarithms.

Tips and pitfalls

- **Never test floats with** `==` after arithmetic: `0.1 + 0.2 == 0.3` is `False` (the sum is `0.30000000000000004`), because 0.1 has no exact binary form. Use `abs(a - b) < tol`.
- `np.spacing(x)` gives the exact gap at x. A relative error near 10^{-16} is as exact as a double gets.